ChessOTB.club — Video Game Recorder: Deep Strategy Document

Version: 1.0 | **Date:** March 2026 | **Author:** Manus AI

Executive Summary

This document provides a complete strategic and technical blueprint for building the **Video Game Recorder** feature on ChessOTB.club — the ability for players to press a single button at their board, record their game on a phone tripod, and receive a full Stockfish analysis review when the game ends. The feature is positioned as the platform's primary growth engine and the central differentiator that transforms ChessOTB.club from a tournament management tool into a comprehensive OTB chess ecosystem.

The core thesis is straightforward: **chess players want to study their OTB games the same way they study online games**, but the friction of manual notation has always blocked this. A phone on a $15 tripod, combined with the right CV pipeline and UX, eliminates that friction entirely.

---

1. The User Experience Flow

1.1 Entry Point: Pairing Card

The feature is surfaced at the most natural moment — when a player receives their pairing. The existing pairing card already shows board number and opponent. Two action buttons sit side by side beneath the pairing: **Chess Clock** (existing) and **Record Game** (new). This placement is deliberate: both actions happen at the same moment in the player's workflow, and grouping them signals that recording is a first-class feature, not an afterthought.

1.2 The Recording Setup Flow (5 Screens)

The setup flow is designed to take under 60 seconds on a first use and under 10 seconds on repeat use. Every screen has a single primary action.

Screen 1 — Permission Gate. If camera permission has not been granted, a full-screen prompt explains why the camera is needed, with a single "Allow Camera" button. This screen is skipped on repeat uses. The copy is specific: *"Your phone's rear camera will record the board. No video is uploaded until your game ends."*

Screen 2 — Orientation Lock. The app detects device orientation and displays a large animated icon prompting the player to rotate their phone to landscape. The screen auto-advances when landscape orientation is confirmed via the `screen.orientation` API. This is non-negotiable: landscape orientation is required for the side-view angle to capture the full board width.

Screen 3 — Camera Preview + Framing Guide. This is the most critical screen. The rear camera (`facingMode: 'environment'`) activates and fills the screen. Overlaid on the live preview is a **board detection overlay** — a translucent green rectangle that the player should align the board corners to. Four corner indicators pulse green when the board is detected, red when it is not. Three status indicators appear at the bottom: **Board Detected** (green/red), **All Corners Visible** (green/red), **Lighting OK** (green/yellow/red). A "Tips" button opens a bottom sheet with the three key setup rules: place phone to the side, use 0.5x zoom if available, ensure the king is visible on the far rank.

The board detection on this screen is **lightweight and client-side** — it does not use the full YOLOv8 pipeline. Instead, it uses a simple OpenCV.js edge detection pass to find the board rectangle. This runs at 5 fps and is fast enough on any modern phone. The goal is not accuracy — it is confidence. The player needs to feel certain their setup will work before they start recording.

Screen 4 — Active Recording. Once the player taps "Start Recording," the UI transitions to a minimal recording screen: a large red pulsing dot with elapsed time, a "Stop Recording" button at the bottom, and a small thumbnail of the live camera feed in the corner (so players can glance and confirm the phone hasn't moved). The screen uses the **Wake Lock API** to prevent the phone from sleeping. The status bar is hidden. The background is near-black to minimize battery drain. A subtle board detection

health indicator (green dot) remains visible — if the board goes out of frame, it turns red with a gentle haptic pulse.

Screen 5 — Processing & Upload. When the player taps “Stop Recording,” the video is finalized and a progress screen appears. The video is chunked and uploaded to S3 in the background. A five-step progress indicator shows: *Saving → Uploading → Queued → Analyzing → Ready*. The player can leave this screen — processing continues in the background and a push notification fires when analysis is ready. If the player stays, the screen auto-navigates to the analysis page when complete.

1.3 The Analysis Page

The analysis page is the chess.com Game Review experience, adapted for OTB. It is described in full in the existing build pack and is already partially implemented. The video recorder flow delivers the player to this page with a fully populated game: PGN, move-by-move Stockfish evaluations, accuracy scores, opening detection, and the Game Highlight card.

---

2. Camera Placement & Physical Setup

The recommended setup — confirmed by ChessCam’s real-world usage data — is a phone placed **to the side of the board on a tripod**, angled to see the full board from a 45–60 degree elevation. This side-view angle is superior to overhead for three reasons: it is easier to mount a phone at table height than directly above a board, it avoids the problem of hands and arms blocking the view during moves, and it produces a perspective that the CV pipeline handles more reliably than a perfectly flat top-down view.

The recommended equipment is a **flexible tripod** (Joby GorillaPod style, ~$15–25) that can be clamped to a table edge or wrapped around a chair back. The platform should recommend this specific form factor in the onboarding tooltip, with a product link if possible.

Setup Variable	Recommended	Acceptable	Will Fail
Camera angle	45–60° elevation, side view	30–70° elevation	Directly overhead, directly side-on
Distance from board	40–70 cm	30–90 cm	<20 cm or >120 cm
Zoom level	0.5x (ultra-wide)	1x	2x+
Lighting	Indirect natural or overhead	Direct lamp	Backlighting, shadows across board
Piece style	Standard Staunton	Most club sets	Novelty/non-standard shapes
Board contrast	High (green/white, brown/cream)	Medium	Low contrast (same-color squares)

---

3. Technical Architecture

3.1 Client-Side Layer (Browser / PWA)

The client is responsible for three things: camera access, recording, and the lightweight framing guide. It does not run the heavy CV pipeline.

Camera access uses `getUserMedia` with `{ video: { facingMode: 'environment', width: { ideal: 1920 }, height: { ideal: 1080 }, frameRate: { ideal: 30 } } }`. The `MediaRecorder` API records to a WebM blob (Chrome/Android) or MP4 (Safari/iOS). Chunked recording uses `timeslice: 5000` on the `MediaRecorder` to fire `ondataavailable` every 5 seconds, enabling progressive upload to S3 via presigned URLs rather than a single large POST at game end.

The framing guide uses **OpenCV.js** (the WebAssembly build, ~8MB, loaded lazily only when the recording screen opens). It processes frames at 5 fps, running Canny edge detection and a simplified Hough line transform to find the board rectangle. The output is a confidence score (0–1) and four corner coordinates, which drive the overlay UI.

3.2 Server-Side Processing Pipeline

The processing pipeline runs as a **Python FastAPI microservice** separate from the main Node.js server. This separation is critical: the CV workload is CPU/GPU-intensive and should not block the main API. The microservice exposes a single internal endpoint: `POST /process` accepts an S3 key and returns a job ID. Processing is asynchronous.

The pipeline has four stages:

Stage 1 — Frame Extraction. FFmpeg extracts frames at 1 fps from the uploaded video. At 1 fps, a 60-minute game produces ~3,600 frames — manageable for processing. Frames are stored temporarily in memory (not written to disk) to minimize I/O.

Stage 2 — Board Localization. OpenCV finds the board in each frame using the same Hough line approach as the client-side framing guide, but with higher accuracy. A homography matrix is computed from the four corner points and applied to produce a normalized top-down 512×512 image of the board. Frames where the board cannot be reliably detected (confidence < 0.7) are skipped. If more than 20% of frames are skipped, the job is flagged for manual review.

Stage 3 — Piece Classification. The normalized board image is divided into 64 equal squares. Each square is passed through a **YOLOv8-nano** model fine-tuned on chess pieces. The model outputs one of 13 classes (empty, wP, wN, wB, wR, wQ, wK, bP, bN, bB, bR, bQ, bK) with a confidence score. The 64 classifications are assembled into a FEN string.

Stage 4 — Move Extraction. Consecutive FEN strings are compared to detect changes. A change is a candidate move. The candidate move is validated using chess.js: if it produces a legal position, it is accepted. If it does not, it is flagged as uncertain and queued for manual correction. The sequence of validated moves is assembled into a PGN.

3.3 Confidence & Fallback System

Every move in the output PGN carries a confidence score. Moves with confidence below 0.85 are flagged. After processing, if fewer than 5 moves are flagged, the game is auto-accepted and Stockfish analysis begins immediately. If 5–15 moves are flagged, the user is prompted to review them in the correction flow. If more than 15 moves are

flagged, the user is offered a fallback to manual PGN entry with the partial reconstruction pre-populated.

Confidence Level	Threshold	Action
High	≥ 0.95	Auto-accepted, no review needed
Medium	0.85–0.95	Accepted, shown in review with yellow flag
Low	0.70–0.85	Flagged for user confirmation
Very Low	< 0.70	Queued for manual correction
Frame Skip	> 20% of frames	Full manual review prompted

3.4 Infrastructure

Component	Technology	Notes
Video storage	AWS S3	Chunked upload via presigned URLs
Processing queue	PostgreSQL `recording_sessions` table	Simple polling, no Redis needed for MVP
CV microservice	Python 3.11 + FastAPI	Deployed as separate container
Board detection	OpenCV 4.x	Hough lines + homography
Piece classification	YOLOv8-nano (Ultralytics)	6MB model, ~50ms per frame on CPU
Frame extraction	FFmpeg	Subprocess call from Python
Chess validation	python-chess	Legal move validation
Analysis engine	Chess-API.com (Stockfish REST)	Existing integration
Push notifications	Web Push (VAPID)	Existing infrastructure

---

4. The Framing Guide — UX Detail

The framing guide is the feature that separates a frustrating experience from a delightful one. Players who set up their phone incorrectly will get bad results. The guide must make correct setup feel effortless.

The overlay consists of a **dashed green rectangle** that represents the target board area. The player physically moves their phone until the board fits inside this rectangle. Four corner circles pulse: they are hollow and red when no board is detected, and fill solid green when a corner is confirmed. A status strip at the bottom of the screen shows three indicators:

The **Board Detected** indicator turns green when the OpenCV pass finds a rectangle with the right aspect ratio (approximately 1:1) and sufficient size (at least 40% of the frame). The **All Corners Visible** indicator turns green when all four corners of the detected rectangle are within the frame bounds with at least 20px margin. The **Lighting OK** indicator analyzes the mean brightness and contrast of the board region — it turns yellow if the board is underexposed (mean brightness < $^{80}/_{255}$) or overexposed (> $^{200}/_{255}$), and red if the variance is too low (suggesting the board is in shadow or the squares are indistinguishable).

The “Start Recording” button is disabled until all three indicators are green. This is a hard gate — not a soft warning. The player cannot start recording with a bad setup. This is the single most important UX decision in the entire feature: it is far better to spend 30 extra seconds on setup than to process a 60-minute video and get an unusable result.

---

5. Phased Build Roadmap

Phase 1 — Foundation (Build Now, 2–3 Weeks)

Phase 1 delivers the complete UX flow with manual PGN entry as the primary input method. The camera recording infrastructure is built but the CV pipeline is not yet connected. Players can record a video (which is uploaded and stored), but the PGN must be entered manually. The analysis page is already built. This phase validates the end-to-end flow and the analysis UX before investing in the CV pipeline.

Deliverables: Camera permission flow, orientation lock, live camera preview screen with framing overlay (client-side OpenCV.js board detection), active recording screen with wake lock, chunked S3 upload, processing status screen, push notification on completion, manual PGN entry fallback.

Phase 2 — CV Pipeline Beta (4–6 Weeks)

Phase 2 connects the Python microservice to the upload pipeline. The YOLOv8 model is trained or fine-tuned on a dataset of standard club chess sets. Processing is triggered automatically after upload. The correction flow is built for flagged moves.

Deliverables: Python FastAPI microservice, FFmpeg frame extraction, OpenCV board localization, YOLOv8 piece classification, move extraction and validation, confidence scoring, correction flow UI, auto-Stockfish trigger on completion.

Phase 3 — Accuracy & Robustness (Ongoing)

Phase 3 is continuous improvement. Every corrected move is a training signal — the correction flow feeds a dataset that improves the model over time. Additional piece styles are added to the training data. The client-side framing guide is tuned based on real-world failure patterns.

Deliverables: Correction data pipeline, model retraining workflow, support for wooden/novelty piece sets, improved lighting handling, multi-board tournament mode (one phone per board).

---

6. The Correction Flow

The correction flow is a critical UX surface that most chess CV tools handle poorly. When the pipeline flags a move as uncertain, the player is shown a dedicated review screen — not a generic error. The screen shows three elements side by side: a still frame from the video at the moment of the uncertain move (the actual video frame, not a reconstruction), the board position as the pipeline understood it, and a list of candidate moves (the 2–3 most likely legal moves given the position, ranked by confidence).

The player taps the correct move. If none of the candidates are correct, a “None of these” option opens a drag-to-move interface on the board. The correction is submitted and the pipeline resumes from that point. Corrections are stored in the `correction_entries` table, which serves as both a user-facing fix and a training data point for future model improvements.

The correction flow is presented as a **post-game review**, not as an interruption during the game. Players complete the game, then review flagged moves in a single batch session. This is a deliberate design choice: interrupting a player mid-game to correct a move detection error would be unacceptable in a competitive setting.

---

7. Key Risks and Mitigations

Risk: Unusual piece sets. Many club sets use non-standard Staunton designs, wooden pieces, or novelty sets. The YOLOv8 model trained on standard sets will perform poorly on these. **Mitigation:** The framing guide includes a piece style selector (Standard / Wooden / Other) that selects the appropriate model variant. In the short term, unusual sets fall back to manual correction.

Risk: Hands blocking pieces. Players frequently place hands on the board while thinking, or pick up pieces and hold them. The pipeline must handle frames where pieces are temporarily absent. **Mitigation:** The move extraction algorithm uses a “stable frame” detector — it only accepts a frame as a candidate move detection point if the frame shows no motion blur and no detected hand (a simple skin-color blob detector). Frames with hands are skipped.

Risk: Camera movement. If the phone is bumped during the game, the homography matrix becomes invalid. **Mitigation:** The pipeline detects sudden shifts in the detected board corners between consecutive frames. If a shift exceeds a threshold, the pipeline re-calibrates the homography from the new position. A warning is added to the game record noting the camera movement event.

Risk: iOS PWA camera limitations. Safari on iOS has known bugs with camera access in `standalone` PWA mode. **Mitigation:** The manifest uses `display: minimal-ui` instead of `standalone` to avoid the WebKit bug. A Safari-specific code path uses `display: browser` as a fallback.

Risk: Upload size. A 60-minute game at 1080p produces approximately 2–4 GB of video. **Mitigation:** The recording uses 720p at 15 fps (sufficient for board detection), reducing file size to ~400–800 MB. Chunked upload via presigned S3 URLs handles large files without timeouts.

---

8. Success Metrics

The feature's success should be measured against three metrics that reflect actual player value, not just usage volume.

Activation rate measures the percentage of tournament pairings where at least one player taps "Record Game." A target of 30% activation within 3 months of launch is realistic for a feature that requires physical setup. **Analysis completion rate** measures the percentage of started recordings that result in a completed Stockfish analysis — the target is 70%, with the remainder falling back to manual PGN entry. **Correction burden** measures the average number of flagged moves per game — the target is fewer than 3 per game, which represents a correction session of under 2 minutes.

---

9. What Makes This Different from ChessCam

ChessCam is a standalone tool that players use independently. ChessOTB.club's recorder is **embedded in the tournament context**, which creates three structural advantages. First, the pairing card entry point means the recording decision happens at the exact right moment — when the player knows their board number and is walking to their seat. Second, the analysis is linked to the tournament record, so accuracy scores appear on the player's profile and contribute to their OTB Accuracy Rating trend. Third, the Game Highlight card is shareable with tournament context — *"Move 23 — I missed mate in 3 in Round 2 of Spring Open 2026"* — which drives social sharing and club awareness in a way that a standalone tool cannot.

The long-term vision is the one articulated in the build pack: **Strava for OTB chess**. Every game recorded builds a personal history. Accuracy trends over time. Opening performance by color. Blunder patterns. This data layer is what transforms ChessOTB.club from a tournament tool into a platform that players return to between tournaments.